

Reviewer Pack — Minimal Rank of Tropicalized Lie Algebras over Communication...

Entry a08138309bb4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 17:42:03 UTC

Minimal Rank of Tropicalized Lie Algebras over Communication Complexity of AND/OR Tree

Entry ID: a08138309bb4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 17:42:03 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Lie Algebras) **Field B** (complexity object): Communication Complexity (AND/OR Trees)

Statement:

{'stmt1': 'For an AND/OR tree representing a communication protocol with depth D and branching factor b , the minimal rank of its associated tropicalized Lie algebra is $\Theta(D^{(3/2)^b}(1/2))$.' , 'stmt2': 'Equivalently, for any AND/OR tree, the number of independent generators required to span its tropicalized Lie algebra is proportional to the square root of the depth and the cube root of the branching factor.' , 'stmt3': 'This invariant holds with high probability for randomly generated AND/OR trees.' }

Rationale (proposer's reasoning):

{'stmt1': 'Tropicalization provides a bridge between algebraic structures like Lie algebras and combinatorial objects like trees, potentially revealing underlying geometric properties of communication complexity.' , 'stmt2': 'Lie algebras are known for their role in describing symmetries and dynamics, which might be intrinsic to the structure of AND/OR trees representing computational processes.' , 'stmt3': 'The combination of depth and branching factor is a fundamental measure of communication complexity, making it a suitable candidate for such an algebraic invariant.' }

Taxonomy category: TROPICAL_LIE_ALGEBRA_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f7e020f454f36f83

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of tropicalized Lie algebras associated with AND/OR trees is within $\pm 10\%$ of $\Theta(D^{(3/2)^b}(1/2))$ for 80% or more of the randomly generated trees.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal rank Tropical Geometry Lie Algebras communication complexity AND/OR tree - tropicalized Lie algebra AND/OR tree depth D branching factor $b^{\theta(D^{3/2})}$ - AND/OR trees tropical geometry generators number proportional square root depth cube root branching factor

Top relevant hits considered: - [http://arxiv.org/abs/math/0702755v1] Restricted simple Lie algebras and their infinitesimal deformations - [http://arxiv.org/abs/1610.00298v4] Khovanskii bases, higher rank valuations and tropical geometry - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2305.01463v2] Test of lepton flavour universality using $B^0 \rightarrow D^{*-} \tau^+ \nu_\tau$ decays with hadronic τ channels - [http://arxiv.org/abs/1101.4605v2] Formulas For The Square Root Modulo p - [http://arxiv.org/abs/1504.00843v9] Generalized Artin Primitive Root Conjecture

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tree(depth, branching_factor):
        if depth == 0:
            return "A"
        else:
            children = [generate_tree(random.randint(1, depth-1), branching_factor) for _ in range(branching_factor)]
            return f"({'', ' '.join(children)})"

    def tropicalize(tree):
        if tree == "A":
            return []
        elif ' ' not in tree:
            return [tree]
```

```

    else:
        children = tree[1:-1].split(',')
        return [tropicalize(child) for child in children]

def compute_rank(tropicalized_tree):
    if not tropicalized_tree:
        return 0
    rank = 0
    for subtree in tropicalized_tree:
        rank += compute_rank(subtree)
    return rank + len(tropicalized_tree)

depths = [5, 10, 15, 20, 30, 40]
branching_factors = [2, 3, 4, 5, 6]
results = []

for depth in depths:
    for _ in range(5): # Ensure at least 5 instances per seed
        tree = generate_tree(depth, random.choice(branching_factors))
        rank = compute_rank(tropicalize(tree))
        expected_rank = math.ceil((depth ** 1.5) * (random.choice(branching_factors) ** 0.5))
        results.append({
            "depth": depth,
            "branching_factor": random.choice(branching_factors),
            "rank": rank,
            "expected_rank": expected_rank
        })

mean_value = sum(result["rank"] for result in results) / len(results)
std_value = math.sqrt(sum((result["rank"] - mean_value) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if abs(result["rank"] - result["expected_rank"]) < std_value) / len(results)

conjecture_holds = support_fraction >= 0.8
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "minimal_rank",
    "metric_value": mean_value,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e4d890b.py", line 82, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e4d890b.py", line 51, in run_trial
    tree = generate_tree(depth, random.choice(branching_factors))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e4d890b.py", line 25, in generate_tree
    children = [generate_tree(random.randint(1, depth-1), branching_factor) for _ in range(branching_factor)]
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e4d890b.py", line 25, in generate_tree
    children = [generate_tree(random.randint(1, depth-1), branching_factor) for _ in range(branching_factor)]
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e4d890b.py", line 25, in generate_tree
    children = [generate_tree(random.randint(1, depth-1), branching_factor) for _ in range(branching_factor)]
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

[Previous line repeated 1 more time]

```

File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")

```

ValueError: empty range in randrange(1, 1)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12687
2	preregistration	ollama_remote	glm4:latest	0	0	6018
3	novelty	ollama_remote	glm4:latest	0	0	5079
4	novelty	ollama_remote	glm4:latest	0	0	6191
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15949
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18411
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14576
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10982
9	judge	ollama_remote	glm4:latest	0	0	17200

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107094 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a08138309bb4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a08138309bb4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a08138309bb4.tar.gz` (if generated)